회로 구성 프로세스

목차

1. 개요
2. BaseCircuitBuilder 초기화
3. Context 생성 및 사용
4. 회로 최적화
5. 증명 생성
6. 관련 문서

개요

이 문서는 halo2-lib에서 `BaseCircuitBuilder` 를 사용하여 영지식증명 회로를 구성하는 전체 프로세스를 설명합니다. 회로 구성은 다음 단계로 이루어집니다:

1. **초기화**: `BaseCircuitBuilder` 생성 및 회로 크기 설정
2. **Context 생성**: 연산을 수행할 가상 영역 생성
3. **연산 수행**: GateChip과 RangeChip을 사용한 제약 조건 생성
4. **최적화**: 회로 파라미터 자동 계산
5. **증명 생성**: MockProver 또는 실제 증명 생성

BaseCircuitBuilder 초기화

BaseCircuitBuilder::new()

`BaseCircuitBuilder` 는 halo2-lib의 핵심 회로 빌더로, 가상 영역을 관리하고 물리적 회로로 매핑하는 역할을 합니다.

소스: `halo2-base/src/gates/circuit/builder.rs`

```
/// BaseCircuitBuilder 생성
/// witness_gen_only: 증명 생성 시에만 true, 키 생성 및 MockProver에서는 false
let mut builder = BaseCircuitBuilder::new(false);
```

witness_gen_only 플래그:

- `true` : 증명 생성(Prover) 전용 모드
- witness 값만 할당하고 제약 조건 정보는 저장하지 않음
- 메모리 효율적이며 실제 증명 생성 시 사용
- `false` : 키 생성(Keygen) 및 검증 모드
- 제약 조건(selector, fixed column, copy constraint) 모두 저장
- MockProver와 키 생성 시 필요

use_k(): 회로 크기 설정

회로의 행(row) 수를 2^k로 설정합니다.

```
let k = 10; // 2^10 = 1024 rows
let mut builder = BaseCircuitBuilder::new(false).use_k(k);
```

k 값의 의미:

- k = 10 → 1,024 rows
- k = 12 → 4,096 rows

- k = 19 → 524,288 rows

k 값이 클수록 더 많은 연산을 수행할 수 있지만, 증명 생성 시간과 메모리 사용량이 증가합니다.

set_lookup_bits(): Lookup 테이블 설정

범위 검사(range check)를 위한 lookup 테이블의 크기를 설정합니다.

```
// lookup 테이블: [0, 2^lookup_bits) 범위의 값
builder.set_lookup_bits(k - 1); // 일반적으로 k-1 사용
```

lookup_bits의 역할:

- lookup 테이블은 [0, 2^lookup_bits) 범위의 모든 값을 포함
- 예: lookup_bits = 9 → [0, 512) 범위
- RangeChip의 범위 검사에 사용됨
- 큰 값일수록 더 넓은 범위를 한 번에 검사 가능하지만 테이블 크기 증가

초기화 코드 예시

소스: `halo2-base/examples/inner_product.rs`, `halo2-base/src/utils/testing.rs`

```
use halo2_base::gates::circuit::builder::BaseCircuitBuilder;

// 1. 기본 초기화
let k = 10;
let mut builder = BaseCircuitBuilder::new(false)
.use_k(k)
.use_lookup_bits(k - 1);
```

```rust
// 2. CircuitBuilderStage를 사용한 초기화
use halo2_base::gates::circuit::CircuitBuilderStage;

// Keygen 단계
let mut builder = BaseCircuitBuilder::from_stage(CircuitBuilderStage::Keygen)
.use_k(k);

// Prover 단계 (witness_gen_only = true)
let mut builder = BaseCircuitBuilder::from_stage(CircuitBuilderStage::Prover)
.use_k(k);

// 3. 사전 계산된 파라미터로 초기화 (Prover 최적화)
use halo2_base::gates::circuit::BaseCircuitParams;

let config_params = BaseCircuitParams {
k: 10,
num_advice_per_phase: vec![4],
num_fixed: 1,
num_lookup_advice_per_phase: vec![1],
lookup_bits: Some(9),
num_instance_columns: 0,
};
```

```
let break_points = vec![/* 사전 계산된 break points */];

let builder = BaseCircuitBuilder::prover(config_params, break_points);
```

Context 생성 및 사용

main() 메서드를 통한 Context 생성

`BaseCircuitBuilder` 의 `main()` 메서드는 특정 phase의 메인 Context를 반환합니다. Context는 회로의 실행 추적(execution trace)을 저장하는 가상 컬럼입니다.

소스: `halo2-base/src/gates/circuit/builder.rs`

```
// phase 0의 메인 Context 가져오기

let ctx = builder.main(0);


// Context에서 witness 값 할당

let a = ctx.load_witness(F::from(5));

let b = ctx.load_witness(F::from(3));
```

Phase 개념

halo2-lib는 다중 phase 증명을 지원합니다. Phase는 증명 생성 시 challenge 값이 필요한 경우 사용됩니다.

Phase 0, 1, 2:

- **Phase 0**: 기본 phase, 대부분의 연산이 여기서 수행됨

- **Phase 1**: Phase 0 이후의 challenge 값을 사용하는 연산
- **Phase 2**: Phase 1 이후의 challenge 값을 사용하는 연산

```
// 각 phase의 Context 생성
let ctx_0 = builder.main(0); // Phase 0
let ctx_1 = builder.main(1); // Phase 1
let ctx_2 = builder.main(2); // Phase 2
```

대부분의 회로는 Phase 0만 사용하며, challenge 값이 필요한 고급 프로토콜(예: Fiat-Shamir 변환)에서만 다중 phase를 사용합니다.

GateChip과 RangeChip 생성

Context에서 연산을 수행하려면 `GateChip` 과 `RangeChip` 이 필요합니다.

```
use halo2_base::gates::{GateChip, RangeChip};
use halo2_base::gates::flex_gate::GateInstructions;
use halo2_base::gates::RangeInstructions;

// GateChip: 기본 산술 연산 (덧셈, 곱셈 등)
let gate = GateChip::default();

// RangeChip: 범위 검사 및 비교 연산
let lookup_bits = builder.lookup_bits().expect("lookup bits not set");
let range = builder.range_chip();
// 또는
```

```rust
let range = RangeChip::new(lookup_bits, builder.lookup_manager().clone());
```

Context에서 연산 수행

소스: halo2-base/examples/inner_product.rs

```rust
use halo2_base::{Context, QuantumCell::Existing};
use halo2_base::gates::flex_gate::{GateChip, GateInstructions};

fn perform_operations<F: ScalarField>(
ctx: &mut Context<F>,
gate: &GateChip<F>,
) {
// 1. Witness 값 할당
let a = ctx.load_witness(F::from(5));
let b = ctx.load_witness(F::from(3));
// 2. 덧셈: a + b
let sum = gate.add(ctx, a, b);
// 3. 곱셈: a * b
let product = gate.mul(ctx, a, b);
// 4. 내적 계산
let a_vec = vec![F::from(1), F::from(2), F::from(3)];
let b_vec = vec![F::from(4), F::from(5), F::from(6)];
let a_assigned = ctx.assign_witnesses(a_vec);
```

```
let b_assigned = ctx.assign_witnesses(b_vec);

// inner_product: a·b = a[0]*b[0] + a[1]*b[1] + a[2]*b[2]

let inner_prod = gate.inner_product(

ctx,

a_assigned.clone(),

b_assigned.into_iter().map(Existing)

);

// 5. 조건부 선택: sel ? a : b

let sel = ctx.load_witness(F::from(1));

let result = gate.select(ctx, a, b, sel);

}
```

여러 Thread 사용

복잡한 회로에서는 여러 thread를 사용하여 병렬로 연산을 수행할 수 있습니다.

```
// 새로운 thread 생성

let ctx_thread_1 = builder.new_thread(0);

let ctx_thread_2 = builder.new_thread(0);


// 각 thread에서 독립적으로 연산 수행

// ...


// 메인 thread로 돌아가기
```

```rust
let ctx_main = builder.main(0);
```

전체 예시

```rust
use halo2_base::gates::circuit::builder::BaseCircuitBuilder;
use halo2_base::gates::{GateChip, GateInstructions};
use halo2_base::halo2_proofs::halo2curves::bn256::Fr;

let k = 10;
let mut builder = BaseCircuitBuilder::new(false)
.use_k(k)
.use_lookup_bits(k - 1);

let gate = GateChip::default();

// Context 가져오기 및 연산 수행
{
let ctx = builder.main(0);
let a = ctx.load_witness(Fr::from(5));
let b = ctx.load_witness(Fr::from(3));
let c = gate.mul(ctx, a, b);
// c = 15 확인
let expected = ctx.load_constant(Fr::from(15));
```

```
ctx.constrain_equal(&c, &expected);

}
```

회로 최적화

calculate_params() 메서드

회로의 모든 연산을 수행한 후, `calculate_params()` 메서드를 호출하여 회로 파라미터를 자동으로 계산합니다.

소스: `halo2-base/src/gates/circuit/builder.rs`, `halo2-base/src/gates/flex_gate/threads/multi_phase.rs`

```
// 회로 파라미터 자동 계산

// minimum_rows: blinding factor를 위한 최소 행 수 (일반적으로 9)

builder.calculate_params(Some(9));
```

Break Points 계산 과정

`calculate_params()` 는 내부적으로 다음을 수행합니다:

1. **통계 수집**: 각 phase별 advice 셀 수와 고정 셀 수 계산
2. **컬럼 수 계산**: 필요한 advice 컬럼과 fixed 컬럼 수 결정
3. **Break Points 설정**: 가상 컬럼을 물리적 컬럼으로 나누는 지점 계산

Break Points:

- 가상 컬럼(virtual column)을 여러 물리적 컬럼(physical column)으로 분할하는 지점
- 각 물리적 컬럼은 최대 `usable_rows` 개의 행을 가질 수 있음

- Break points는 증명 생성 시 재사용되어 동일한 회로 구조를 보장

```
// Break points 가져오기
let break_points = builder.break_points();


// Break points를 사용하여 Prover 생성
let prover_builder = BaseCircuitBuilder::prover(config_params, break_points);
```

usable_rows 설정

usable_rows는 실제로 witness 할당에 사용할 수 있는 행의 수입니다.

```
let k = 10; // 2^10 = 1024 total rows
let minimum_rows = 9; // blinding factors
let usable_rows = (1 << k) - minimum_rows; // 1024 - 9 = 1015 rows
```

계산 공식:

```
usable_rows = 2^k - minimum_rows
```

- `minimum_rows` : 영지식 속성을 보장하기 위한 blinding factor 행
- 일반적으로 9개 행을 사용 (halo2 기본값)

최적화 전략

1. Advice 컬럼 수 계산:

```
// 각 phase별 필요한 advice 컬럼 수
num_advice_per_phase[i] = ceil(total_advice_cells[i] / usable_rows)
```

2. Fixed 컬럼 수 계산:

```
// 고유한 상수 값의 수에 따라 결정
num_fixed = ceil(total_unique_constants / 2^k)
```

3. Lookup Advice 컬럼 수 계산:

```
// 범위 검사가 필요한 셀의 수
num_lookup_advice_per_phase[i] = ceil(total_lookup_cells[i] / usable_rows)
```

최적화 예시

```
use halo2_base::gates::circuit::builder::BaseCircuitBuilder;
use halo2_base::gates::{GateChip, GateInstructions};
use halo2_base::halo2_proofs::halo2curves::bn256::Fr;

let k = 12;
let mut builder = BaseCircuitBuilder::new(false)
.use_k(k)
.use_lookup_bits(k - 1);
```

```
let gate = GateChip::default();


// 많은 연산 수행
{
let ctx = builder.main(0);
for i in 0..1000 {
let a = ctx.load_witness(Fr::from(i));
let b = ctx.load_witness(Fr::from(i + 1));
let _ = gate.mul(ctx, a, b);
}
}


// 파라미터 자동 계산
let config_params = builder.calculate_params(Some(9));


// 계산된 파라미터 확인
println!("k: {}", config_params.k);
println!("num_advice_per_phase: {:?}", config_params.num_advice_per_phase);
println!("num_fixed: {}", config_params.num_fixed);
println!("num_lookup_advice_per_phase: {:?}", config_params.num_lookup_advice_pe
```

출력 예시:

```
Gate Chip | Phase 0: 3000 advice cells
```

```
Total 15 fixed cells

k: 12

num_advice_per_phase: [1]

num_fixed: 1

num_lookup_advice_per_phase: [0]
```

증명 생성

MockProver 사용법

개발 단계에서는 `MockProver` 를 사용하여 회로의 제약 조건을 빠르게 검증할 수 있습니다.

소스: `halo2-base/src/utils/testing.rs`

```rust
use halo2_base::halo2_proofs::dev::MockProver;


// MockProver 실행
MockProver::run(k  as u32, &builder, vec![])
.unwrap()
.assert_satisfied();
```

MockProver의 특징:

- 실제 증명을 생성하지 않고 제약 조건만 검증
- 매우 빠른 실행 속도 (실제 증명 생성의 1/100 이하)
- 개발 및 디버깅에 적합

- 제약 조건 위반 시 상세한 오류 메시지 제공

실제 증명 생성 (create_proof)

실제 증명을 생성하려면 다음 단계를 거칩니다:

1. SRS(Structured Reference String) 생성:

```
use halo2_base::utils::fs::gen_srs;
```

```
let params = gen_srs(k); // KZG commitment scheme 파라미터
```

2. Verification Key 생성:

```
use halo2_base::halo2_proofs::plonk::keygen_vk;
```

```
let vk = keygen_vk(&params, &builder).unwrap();
```

3. Proving Key 생성:

```
use halo2_base::halo2_proofs::plonk::keygen_pk;
```

```
let pk = keygen_pk(&params, vk, &builder).unwrap();
```

4. 증명 생성:

```
use halo2_base::halo2_proofs::plonk::create_proof;
use halo2_base::halo2_proofs::poly::kzg::commitment::KZGCommitmentScheme;
```

```rust
use halo2_base::halo2_proofs::poly::kzg::multiopen::ProverSHPLONK;
use halo2_base::halo2_proofs::transcript::{Blake2bWrite, Challenge255};

let mut transcript = Blake2bWrite::<_, _, Challenge255<_>>::init(vec![]);
create_proof::<KZGCommitmentScheme<Bn256>, ProverSHPLONK<'_, Bn256>, _, _, _, _>
&params,
&pk,
&[builder],
&[&[]], // public instances
rng,
&mut transcript,
).expect("proof generation should not fail");

let proof = transcript.finalize();
```

5. 증명 검증:

```rust
use halo2_base::halo2_proofs::plonk::verify_proof;
use halo2_base::halo2_proofs::poly::kzg::multiopen::VerifierSHPLONK;
use halo2_base::halo2_proofs::poly::kzg::strategy::SingleStrategy;
use halo2_base::halo2_proofs::transcript::Blake2bRead;

let verifier_params = params.verifier_params();
let strategy = SingleStrategy::new(&params);
```

```
let mut transcript = Blake2bRead::<_, _, Challenge255<_>>::init(&proof[..]);


verify_proof::<KZGCommitmentScheme<Bn256>, VerifierSHPLONK<'_, Bn256>, _, _, _>(
verifier_params,
pk.get_vk(),
strategy,
&[&[]], // public instances
&mut transcript,
).unwrap();
```

inner_product.rs 예제 기반 전체 흐름

소스: `halo2-base/examples/inner_product.rs`, `halo2-base/src/utils/testing.rs`

다음은 내적(inner product) 계산 회로의 전체 구현 예시입니다:

```
use halo2_base::gates::circuit::builder::BaseCircuitBuilder;
use halo2_base::gates::circuit::CircuitBuilderStage;
use halo2_base::gates::{GateChip, GateInstructions, RangeChip};
use halo2_base::halo2_proofs::halo2curves::bn256::Fr;
use halo2_base::halo2_proofs::dev::MockProver;
use halo2_base::{Context, QuantumCell::Existing};


// 내적 계산 함수
fn inner_product_circuit<F: ScalarField>(
```

```
ctx: &mut Context<F>,

gate: &GateChip<F>,

a: Vec<F>,

b: Vec<F>,

) -> AssignedValue<F> {

assert_eq!(a.len(), b.len());

// Witness 값 할당

let  a_assigned = ctx.assign_witnesses(a);

let  b_assigned = ctx.assign_witnesses(b);

// 내적 계산: a·b = a[0]*b[0] + a[1]*b[1] + ... + a[n-1]*b[n-1]

gate.inner_product(

ctx,

a_assigned,

b_assigned.into_iter().map(Existing)

)

}



fn  main() {

let  k = 12;

// === 1. Keygen 단계 ===

let  mut  builder = BaseCircuitBuilder::from_stage(CircuitBuilderStage::Keygen)

.use_k(k)

.use_lookup_bits(k - 1);
```

```rust
let gate = GateChip::default();
// 초기 입력으로 회로 구조 결정
{
let ctx = builder.main(0);
let a = vec![Fr::from(0); 5];
let b = vec![Fr::from(0); 5];
inner_product_circuit(ctx, &gate, a, b);
}
// 회로 파라미터 계산
let config_params = builder.calculate_params(Some(9));
// MockProver로 검증
MockProver::run(k as u32, &builder, vec![])
.unwrap()
.assert_satisfied();
// === 2. Setup 단계 ===
use halo2_base::utils::fs::gen_srs;
use halo2_base::halo2_proofs::plonk::{keygen_vk, keygen_pk};
let params = gen_srs(k);
let vk = keygen_vk(&params, &builder).unwrap();
let pk = keygen_pk(&params, vk, &builder).unwrap();
// Break points 저장
let break_points = builder.break_points();
// === 3. Prover 단계 ===
```

```rust
let mut prover_builder = BaseCircuitBuilder::prover(
config_params.clone(),
break_points
);
let gate = GateChip::default();
// 실제 입력으로 증명 생성
{
let ctx = prover_builder.main(0);
let a = vec![Fr::from(1), Fr::from(2), Fr::from(3), Fr::from(4), Fr::from(5)];
let b = vec![Fr::from(6), Fr::from(7), Fr::from(8), Fr::from(9), Fr::from(10)];
let result = inner_product_circuit(ctx, &gate, a, b);
// result = 1*6 + 2*7 + 3*8 + 4*9 + 5*10 = 130
}
// 증명 생성
use halo2_base::halo2_proofs::plonk::create_proof;
use halo2_base::halo2_proofs::poly::kzg::commitment::KZGCommitmentScheme;
use halo2_base::halo2_proofs::poly::kzg::multiopen::ProverSHPLONK;
use halo2_base::halo2_proofs::transcript::{Blake2bWrite, Challenge255};
use halo2_base::halo2_proofs::halo2curves::bn256::Bn256;
use rand::rngs::StdRng;
use rand::SeedableRng;
let rng = StdRng::seed_from_u64(0);
let mut transcript = Blake2bWrite::<_, _, Challenge255<_>>::init(vec![]);
```

```
create_proof::<
KZGCommitmentScheme<Bn256>,
ProverSHPLONK<'_, Bn256>,
Challenge255<_>,
_,
Blake2bWrite<Vec<u8>, _, _>,
_,
>(&params, &pk, &[prover_builder], &[&[]], rng, &mut transcript)
.expect("proof generation should not fail");
let proof = transcript.finalize();
println!("Proof size: {} bytes", proof.len());
// === 4. Verifier 단계 ===
use halo2_base::halo2_proofs::plonk::verify_proof;
use halo2_base::halo2_proofs::poly::kzg::multiopen::VerifierSHPLONK;
use halo2_base::halo2_proofs::poly::kzg::strategy::SingleStrategy;
use halo2_base::halo2_proofs::transcript::Blake2bRead;
let verifier_params = params.verifier_params();
let strategy = SingleStrategy::new(&params);
let mut transcript = Blake2bRead::<_, _, Challenge255<_>>::init(&proof[..]);
verify_proof::<
KZGCommitmentScheme<Bn256>,
VerifierSHPLONK<'_, Bn256>,
Challenge255<_>,
```

```
Blake2bRead<&[u8], _, _>,
SingleStrategy<'_, Bn256>,
>(verifier_params, pk.get_vk(), strategy, &[&[]], &mut  transcript)
.expect("verification should succeed");
println!("Verification successful!");
}
```

단계별 요약

1. Keygen (키 생성):

- `witness_gen_only = false` 로 builder 생성
- 더미 입력으로 회로 구조 결정
- `calculate_params()` 로 회로 파라미터 계산
- Verification Key와 Proving Key 생성
- Break points 저장

2. Prover (증명 생성):

- `BaseCircuitBuilder::prover()` 로 최적화된 builder 생성
- 저장된 break points 사용
- 실제 입력으로 witness 생성
- `create_proof()` 로 증명 생성

3. Verifier (검증):

- Verification Key 사용
- `verify_proof()` 로 증명 검증

- Public instances 확인

성능 최적화 팁

1. witness_gen_only 플래그 활용:

```
// Keygen: 모든 정보 저장
let keygen_builder = BaseCircuitBuilder::new(false);

// Prover: witness만 생성 (메모리 효율적)
let prover_builder = BaseCircuitBuilder::new(true);
```

2. Break points 재사용:

```
// Keygen 시 break points 저장
let break_points = keygen_builder.break_points();

// Prover 시 재사용
let prover_builder = BaseCircuitBuilder::prover(config_params, break_points);
```

3. 적절한 k 값 선택:

- 너무 작으면: 회로가 맞지 않음
- 너무 크면: 불필요한 메모리와 시간 소비
- `calculate_params()` 출력을 보고 조정

4. Lookup 테이블 최적화:

- 범위 검사가 없으면 `lookup_bits = None` 설정

- 필요한 최소 크기만 사용

관련 문서

- 02-핵심-개념.md: Context, AssignedValue, QuantumCell 상세 설명
- 04-제약-조건-시스템.md: GateChip과 RangeChip 사용법
- 05-가상-영역-관리.md: Break points와 가상-물리 매핑 상세

다음 단계

회로 구성 방법을 익혔다면, 이제 제약 조건을 생성하는 게이트 시스템을 학습하세요:

→ **04. 제약 조건 시스템**: FlexGate와 RangeGate를 통한 제약 조건 구현

참고 자료

- **소스 파일**:
- `halo2-base/src/gates/circuit/builder.rs` : BaseCircuitBuilder 구현
- `halo2-base/src/gates/circuit/mod.rs` : BaseConfig 및 Circuit trait 구현
- `halo2-base/src/gates/flex_gate/threads/multi_phase.rs` : calculate_params 구현
- `halo2-base/examples/inner_product.rs` : 전체 예제
- `halo2-base/src/utils/testing.rs` : 테스트 유틸리티
- **외부 링크**:
- Halo2 Book: Halo2 공식 문서
- Axiom Crypto GitHub: halo2-lib 저장소